Sistemi operativi II

Anno 2021

Valentino Bocchetti

Contents

1 Introduzione al corso **2**

2 Lezione del 08-03 **3**

2.1 I sistemi operativi 3

2.2 Sistemi mono e multi programmati 3

2.3 Sistemi paralleli 4

2.4 Sistemi distribuiti 4

2.5 Ambiente di elaborazione (Client-Server o Peer-to-Peer) 4

2.6 Cloud computing 5

2.7 Sistemi mobile 5

2.8 Avvio di un sistema operativo nel dettaglio 5

2.9 Tabella degli interrupt 6

2.10 Memoria centrale 6

2.11 Memoria cache 6

2.12 Memoria di massa 6

2.13 I processi 7

2.14 Activity 8

2.15 Politiche utilizzate dallo scheduler 9

2.16 Sistema mono e multi tasking 9

2.17 Throughput (utilizzo della CPU) 10

2.18 Overhead 10

1 Introduzione al corso

`Ricevimento` (da concordare almeno 24 ore prima) → Venerdì 10-12 (chat privata su teams)

`Email` → silviobarra@unina.it con oggetto **[INFO-S02]** (iscritto al *anno di corso* e *corso di laurea* SO2 esame facoltativo o obbligatorio)

Obiettivi

- Scelte progettuali ed implementative di un reale sistema operativo a base linux
- Sistemi distribuiti e del cloud computing
- Sistemi operativi ad uso mobile
- IoT

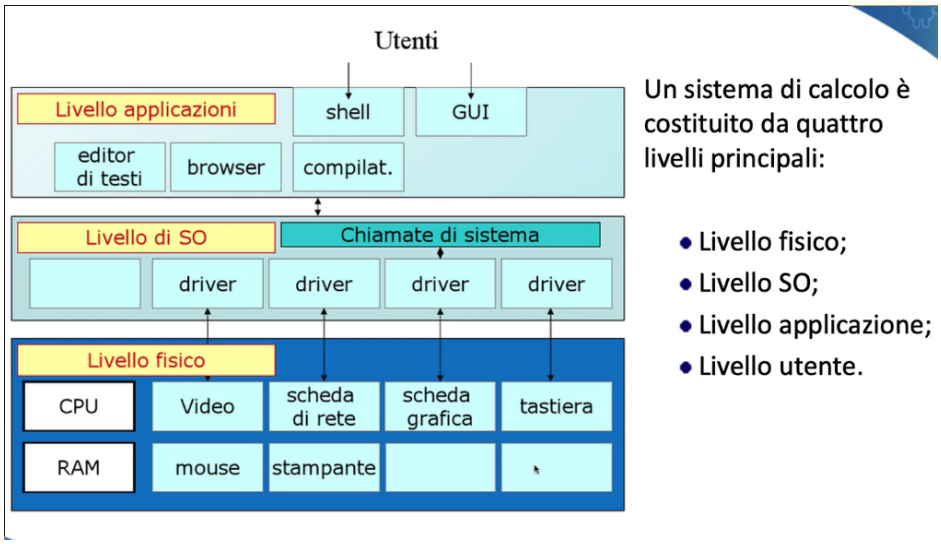
Prove intercorso:

- Domande sul sistema linux
- Domande sui sistemi mobile

2 Lezione del 08-03

2.1 I sistemi operativi

Le componenti di un sistema di calcolo è così definita:



Un **sistema operativo** è difficile da definire (in base alle grandi variazioni che presenta). Una definizione è quella di definirlo come un insieme di software atti a gestire l'hardware sottostante.

Può essere visto come un allocatore di risorse:

- memoria;
- dispositiv di `I/O`;
- processi

Può essere visto come un programma di controllo:

- gestore dei programmi di sistema ed applicativi;
- verifica del corretto funzionamento dei dispositivo di `I/O`

2.2 Sistemi mono e multi programmati

Un sistema è detto **mono-programmato** quando in ogni momento è presente una sola **immagine** (pacchetto processo + dati)

Con l'introduzione del concetto di **spooling**, nascono i primi sistemi **multi-programmati** (il sistema è in grado di eseguire più job in concorrenza). Ciò comporta di tener conto della gestione delle risorse ad essi dedicate:

- Tempo di calcolo
- Memoria
- `I/O`

In questi sistemi si ha il aso di sistemi multi-utenti. Di conseguenza nascono i sistemi **time-sharing** (nascita dei primi PC). Ogni processo utilizza periodicamente un intervallo di tempo prestabilito (un **quanto**)

2.3 Sistemi paralleli

Si arriva alla nascita dei sistemi **Paralleli** dotati di più CPU :

- Symmetric multiprocessing (SMP) → ogni processore esegue una copia identica del SO
- Asymmetric multiprocessing (AMP) → a ciascun processore è assegnato un task specifico; il processore **master** schedua e allora il lavoro per i processori **slave**

2.4 Sistemi distribuiti

Ripartiscono l'elaborazione su diversi processori. Ciascun processore ha una memoria locale e i processori comunicano con gli altri attraverso di comunicazione quali bus ad alta velocità o rete. La rete può essere:

- Locale (LAN);
- Geografica (WAN);
- Metropolitana (MAN);
- Small (SAN).

2.5 Ambiente di elaborazione (Client-Server o Peer-to-Peer)

Esistono due principali tipi di server:

- server per l'elaborazione;

- file server

Nei sistemi Peer-to-Peer invece ciascun pc può essere server o client

- Il carico è suddiviso su più server (si evitano i sovraccarichi);
- Necessita una politica di gestione
- Nuova entità (**load balancer**)

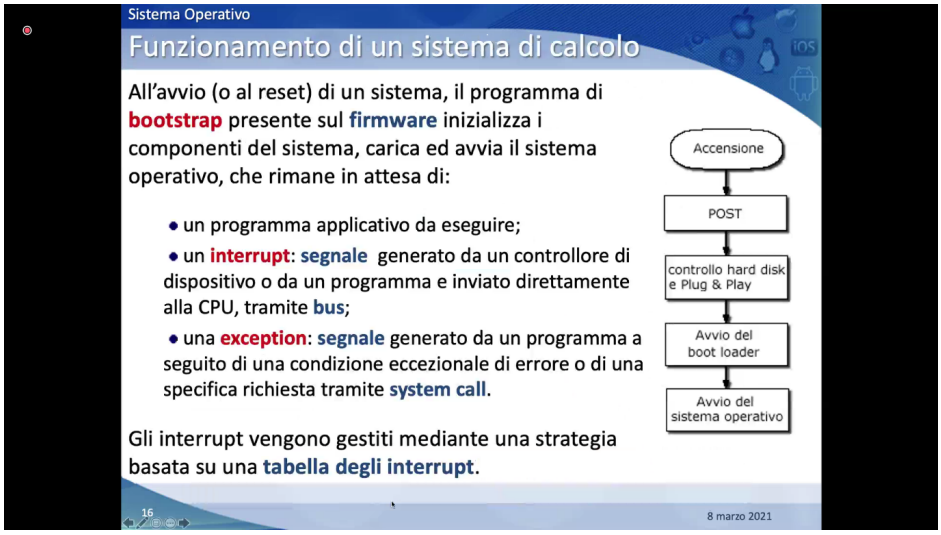
2.6 Cloud computing

Paradigma di uso delle risorse computazionali attraverso il cloud (memorizzazione, archiviazione ed elaborazione dati)

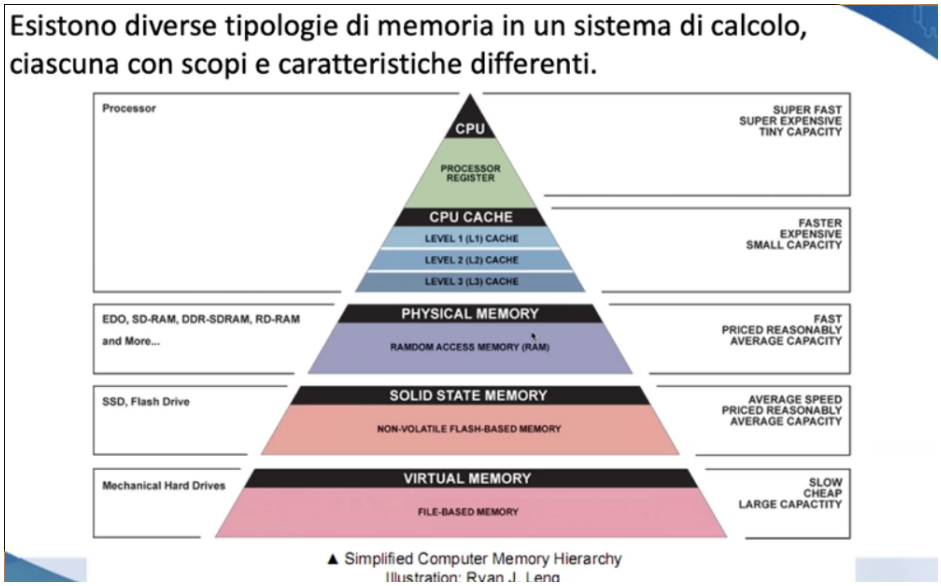
2.7 Sistemi mobile

Il sistema operativo in questo caso ha orientamento specifico (a causa delle dimensioni del dispositivo)

2.8 Avvio di un sistema operativo nel dettaglio



2.9 Tabella degli interrupt



2.10 Memoria centrale

È rappresentata ccome un vettore di parole machina dalle centinaia di migliaia all'ordine dei miliardi di parole. Nella memoria centrale vengono caricati programmi e i dati

2.11 Memoria cache

Memoria molto veloce con lo scopo di ridurre al minimo gli accessi ad unità di memoria più lente. Bisogna garantire la coerenza delle informazioni memorizzate in una o più unità di memoria cache (il cui grado di complessità aumenta a seconda del tipo di sistema)

2.12 Memoria di massa

Divise in secondarie (dischi magnetici, solidi) e terziarie (dischi ottici, CD, DVD, nastri). Le responsabilità del SO rispetto alla gestione delle memorie di massa sono:

- gestione dello spazio libero;
- assegnazione dello spazio;
- scheduling del disco.

I dati vengono scritti nelle memoria mediante file

2.13 I processi

Un processo può essere definito come un programma di elaborazione o **job**. Un processo è un entità attiva, dotata di un program counter. La sua esecuzione è sequenziale e necessita di determinate risorse:

- CPU
- memoria
- dispositivi `I/O`;
- file

Il SO deve garantire alcune operazioni fondamentali per i processi:

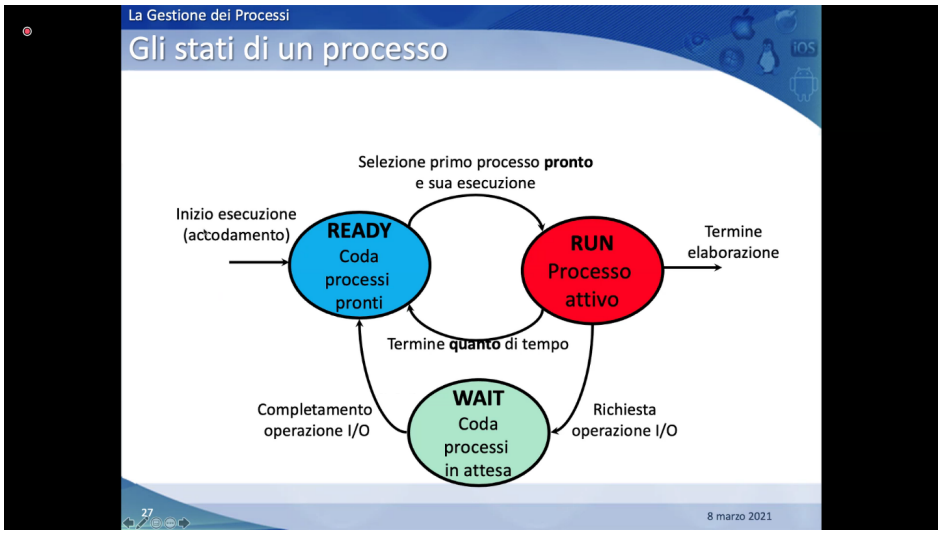
- creazione e cancellazione di processi;
- sospensione e ripristino di processi;
- sincronizzazione di processori;
- comunicazione fra processi;
- soluzioni per lo stato di **deadlock** di processi (questo compito spetta al kernel attraverso lo **scheduler**)

Gli stati di un processo

Ci si può trovare in 2 stati:

- attesa;
- pronto;

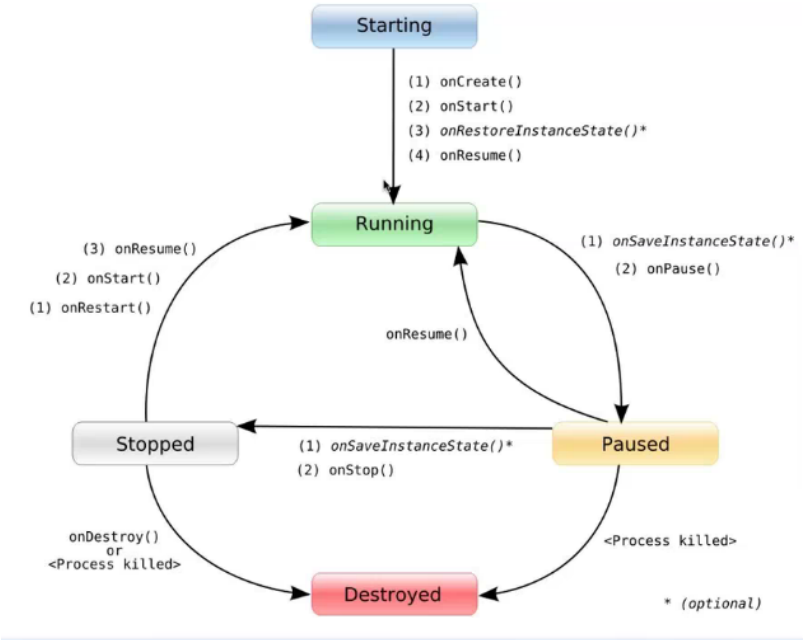
In ogni instante di tempo vi è un solo stato attivo



2.14 Activity

Una **Activity** è caratterizzata da un ciclo di vita ed in ogni istante si trova in uno dei seguenti stati:

- `Active/Resumed` (running) → essa è visibile e può ricevere input dall'utente;
- Paused → essa è parzialmente visibile a causa di un'altra activity in foreground, non può ricevere input dall'utente (*es* è in attesa a causa di un alert dialog presente al di sopra di essa);
- Stopped → essa non è visibile (resta in background)

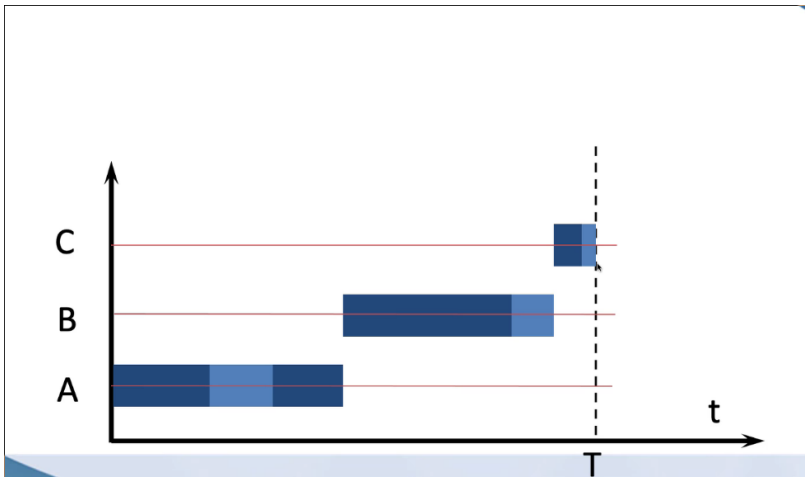


2.15 Politiche utilizzate dallo scheduler

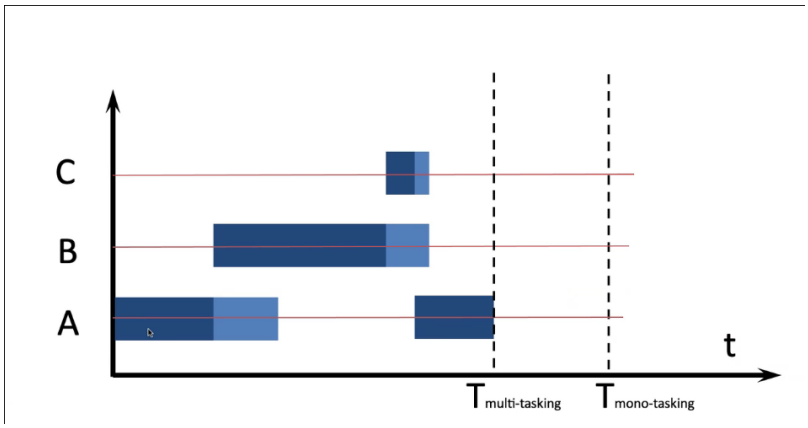
- **Preemtive** → la CPU in uso può essere tolta e passata a un altro in qualsiasi momento;
- **Non Preemtive** → una volta che il processo ha ottenuto l'us della CPU non può essere interrotta fino a che lui stesso non la rilascia;

2.16 Sistema mono e multi tasking

La gestione della CPU (mantenere in maniera efficiente il suo uso) è un obiettivo del sistema operativo. Nel caso di un sistema mono-tasking avremo:



Nel caso di un sistema multi-tasking avremo:



(*In blu scuro il tempo di utilizzo CPU e in chiaro il tempo di attesa di eventi esterni*)

2.17 Throughput (utilizzo della CPU)

Si ha dal rapporto di Tp/Tt dove:

- Tp = tempo dedicato dalla CPU alla esecuzione dei processi utente;
- Tt = tempo totale di osservazione

Si ha un'alta efficienza con un throughput tendente ad 1

2.18 Overhead

Tempo impiegato dal SO per trasferire il controllo da un programma ad un altro